

FdC Railway Manager

Documentazione modulare

Progetto FDC-App

Luglio 2026 — v0.1

Indice

I	Fondamenti	9
1	Prefazione	11
1.1	Scopo del documento	11
1.2	Pubblico e prerequisiti	11
1.3	Convenzioni	11
1.4	Versione	11
2	Architettura generale	13
2.1	Visione	13
2.2	Struttura repository	13
2.3	Refactor Strangler (Fasi 0–4)	13
2.4	Swift Packages (scaffold)	13
2.5	Test	13
2.6	Feature flags	14
II	Moduli core	15
3	Modulo FDCDomain	17
3.1	Responsabilità	17
3.2	Grafo del dominio	17
3.3	Tipi per categoria	17
3.3.1	Topologia	17
3.3.2	Infrastruttura di dettaglio	18
3.3.3	Servizio e rotazione	18
3.4	Package SPM	18
3.5	Regole di evoluzione	18
4	Modulo FDCScheduling	19
4.1	Responsabilità	19
4.2	Componenti principali	19
4.3	Pipeline a 8 step	19
4.3.1	Dettaglio step	19
4.4	Entry point	20
4.5	Dipendenze esterne (adapter app)	20
4.6	Test di regressione	20
4.7	Storico	20
5	Modulo Pathfinding	23
5.1	Responsabilità	23
5.2	API principale	23
5.3	Integrazione	23

5.4	Vincoli algoritmici	23
5.5	Test	24
5.6	Evoluzione (Fase 6+)	24
6	Integrazione Swift Package	25
6.1	FDCDomain nel target Xcode	25
6.2	Decoupling UI (Fase 6)	25
6.3	API pubbliche	25
6.4	Test	26
6.5	Prossimi passi (Fase 8)	26
7	NetworkModel come adapter UI	27
7.1	Obiettivo (Fase 7)	27
7.2	NetworkTopologyService	27
7.3	NetworkModel	27
7.4	DomainBridge	27
7.5	TrainCategory e DTO	28
7.6	Test	28
7.7	Prossimi passi (Fase 8)	28
8	Package FDCScheduling	29
8.1	Obiettivo (Fase 8)	29
8.2	Architettura dipendenze	29
8.3	Contenuto package	29
8.4	Protocolli (FDCDomain)	29
8.5	Shell app	29
8.6	Test	30
8.7	Nota tecnica	30
A	Indice dei sorgenti	31

Elenco delle figure

2.1	Direzione delle dipendenze (semplificata).	13
3.1	Relazioni principali tra tipi di dominio.	17
4.1	<code>ScheduleOptimizationPipeline.execute</code>	20
8.1	Dipendenze Fase 8: package puro + adapter nell'app.	29

Elenco delle tabelle

2.1	Pacchetti SPM locali (<code>Packages/</code>).	14
4.1	Servizi di scheduling.	19
4.2	Suite test scheduling (<code>FdC Railway ManagerTests</code>).	21
5.1	Funzioni <code>NetworkPathfinder</code>	23
6.1	Spostamenti layer UI.	25
8.1	Boundary tra package e app.	30

Parte I

Fondamenti

Capitolo 1

Prefazione

1.1 Scopo del documento

Questo manuale descrive i **moduli software** del progetto *FdC Railway Manager*: architettura, responsabilità, dipendenze e flussi dati. È pensato per sviluppatori che contribuiscono al refactor verso un *functional core* testabile.

La documentazione viene estesa **incrementalmente**: ogni capitolo corrisponde a un modulo (o sotto-modulo) e può essere aggiornato in autonomia senza riscrivere l'intero volume.

1.2 Pubblico e prerequisiti

- Conoscenza di Swift 5.9+ e SwiftUI.
- Familiarità con pattern *Strangler*, CQS e DI esplicita.
- Nozioni base di ferrovie: stazione, binario, Taktfahrplan.

1.3 Convenzioni

Domain/Model/ Tipi di dominio condivisi; preferibilmente solo **Foundation** (o **CoreLocation** dove necessario).

Services/ Logica applicativa orchestrata dai ViewModel.

Models/ Adapter, **NetworkModel**, estensioni UI.

Pipeline Sequenza deterministica di trasformazioni su **[Train]**.

1.4 Versione

Versione documento	0.1 (Fase 4 completata)
Ultimo aggiornamento	Luglio 2026
Commit di riferimento	00db4c2

Capitolo 2

Architettura generale

2.1 Visione

L'applicazione è organizzata in tre aree funzionali indipendenti (**Editor**, **Scheduler**, **Simulator**) che comunicano esclusivamente tramite tipi in `Domain/Model/`. Le dipendenze puntano sempre verso il dominio (regola delle frecce).

2.2 Struttura repository

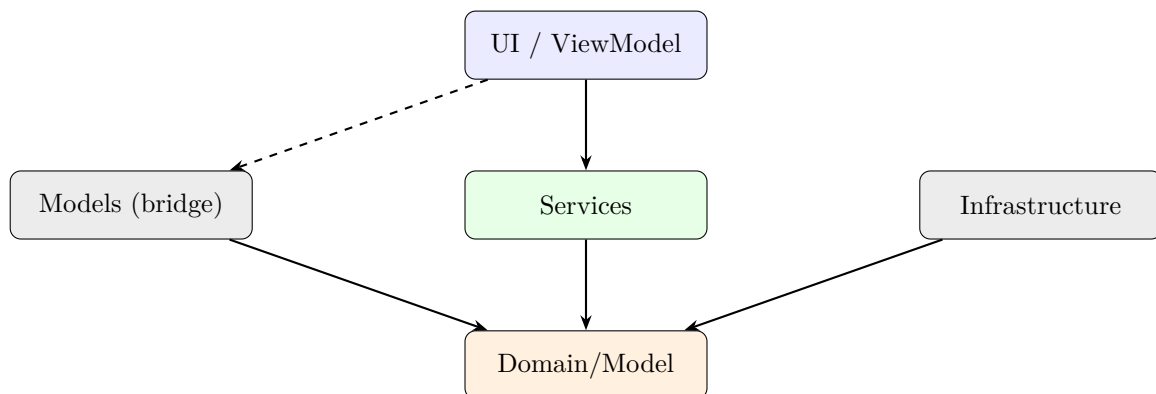


Figura 2.1: Direzione delle dipendenze (semplificata).

2.3 Refactor Strangler (Fasi 0–4)

Il monolite `RailwayScheduleOptimizer` è stato sostituito da `Services/Scheduling/`. La migrazione dei modelli da `Models/` a `Domain/Model/` è in corso: i tipi infrastrutturali (`Node`, `Edge`, `Signal`, ...) risiedono già nel dominio; `NetworkModel` e i DTO restano nel layer bridge.

2.4 Swift Packages (scaffold)

2.5 Test

- 20 test `XCTest` nel target `FdC Railway ManagerTests`.
- 2 test nel package `FDCDomain`.

Tabella 2.1: Pacchetti SPM locali (`Packages/`).

Package	Stato	Note
FDCDomain	Compilabile	9 tipi Foundation via symlink
FDCScheduling	Scaffold	Non linkato al target Xcode

- I servizi di scheduling sono `struct` pure su `RailwayTopology` (no `@MainActor`, no singleton).

2.6 Feature flags

Funzionalità sperimentali in `FeatureFlags.swift`: `simulatorGameMode`, `realtimeConflictHighlight`. Il flag `useLegacyScheduleOptimizer` è stato rimosso con la Fase 4.

Parte II

Moduli core

Capitolo 3

Modulo FDCDomain

3.1 Responsabilità

FDCDomain contiene i **tipi di dominio ferroviario**: entità immutabili (o quasi), **Codable** per persistenza, senza dipendenze da **SwiftUI** o da **NetworkModel**.

Il package SPM in **Packages/FDCDomain/** espone un sottoinsieme **Foundation-only** compilabile standalone; il target Xcode include l'intera cartella **FdC Railway Manager/Domain/Model/**.

3.2 Grafo del dominio

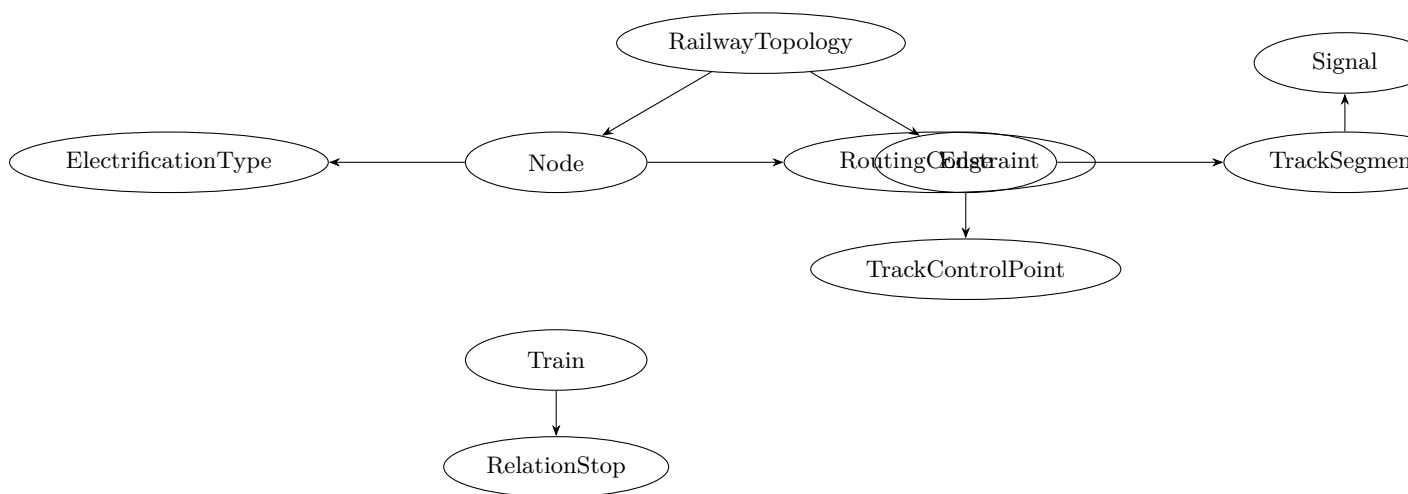


Figura 3.1: Relazioni principali tra tipi di dominio.

3.3 Tipi per categoria

3.3.1 Topologia

RailwayTopology Snapshot immutabile di **nodes** e **edges**. Delega pathfinding a **NetworkPathfinder** (Fase 5). Nessun **init(network:)**.

Node Stazione, deposito, interscambio; vincoli di instradamento, altitudine, parametri Takt (**taktMinutes**).

Edge Tratta tra due nodi; segmenti **TrackSegment**, punti di controllo, elettrificazione, limite velocità.

3.3.2 Infrastruttura di dettaglio

TrackSegment Binario fisico con ordine, lunghezza, segnale opzionale, occupazione.

TrackControlPoint Punto sulla tratta (lat/lon/alt).

Signal Segnale con aspetto (`stop`, `proceed`, `caution`).

Switch Deviazione con `nodeId` e collegamenti.

ElectrificationType Trazione: diesel, 3 kV, 25 kV, 950 V.

RoutingConstraint Vincoli di percorso per stazione.

3.3.3 Servizio e rotazione

Train Treno con fermate `RelationStop`, orari, assegnazione binari, `routeId`.

RelationStop Fermata o transito su una stazione.

Vehicle Rotazione materiale rotabile.

TrainRoute Template di percorso (origine, destinazione, sequenza stazioni). `displayColor` in `Models/TrainRoute+UI.swift`.

3.4 Package SPM

```
Packages/FDCDomain/  
  Package.swift  
  Sources/FDCDomain/    # symlink ai .swift Foundation-only  
  Tests/FDCDomainTests/
```

Tipi inclusi nel package (v0.1): `ElectrificationType`, `Signal`, `Switch`, `RoutingConstraint`, `RelationStop`, `Vehicle`, `TrackSegment`, `TrackControlPoint`, `RailwayConstants`.

Tipi **esclusi** fino a decoupling UI / NetworkModel: `Node`, `Edge`, `Train`, `RailwayTopology`, `LineProfilePoint`, `TrainRoute`, `Ferrovia`.

3.5 Regole di evoluzione

1. Nuovi modelli → `Domain/Model/`, solo `import Foundation`.
2. Estensioni SwiftUI → file `+UI.swift` in `Models/`.
3. Rimuovere voci duplicate da `project.pbxproj` (sync group).
4. Aggiungere test in `FDCDomainTests` quando il tipo è standalone.

Capitolo 4

Modulo FDCScheduling

4.1 Responsabilità

FDCScheduling orchestrate la **generazione e ottimizzazione degli orari**. Il codice sorgente risiede in `FdC Railway Manager/Services/Scheduling/`; il package `SPM Packages/FDCScheduling/` è linkato al target Xcode (Fase 8).

4.2 Componenti principali

Tabella 4.1: Servizi di scheduling.

Tipo	Ruolo
<code>ScheduleGenerationEngine</code>	Orchestratore: crea treni grezzi, invoca pipeline, gestisce modalità Takt/libera.
<code>ScheduleOptimizationPipeline</code>	Pipeline a 8 step (vedi §4.3).
<code>TaktEngine</code>	Allineamento Taktfahrplan, diagnostica hub.
<code>ScheduleRefresher</code>	Ricalcolo orari fisici (arrivi/partenze).
<code>ScheduleAIResolver</code>	Step AI cloud (<code>RailwayAIService</code>).
<code>KinematicCalculator</code>	Tempi di percorrenza su topologia.
<code>PathResolver</code>	Risoluzione percorsi tra stazioni.
<code>VehicleSuitabilityEngine</code>	Compatibilità rotabile/linea.

4.3 Pipeline a 8 step

4.3.1 Dettaglio step

1. **Ottimizzazione partenze** — Per ogni nuovo treno, prova shift coarse ($\pm 10, 20, 30, 50$ min) e fine ($\pm 1, 2, 5$ min); minimizza i conflitti rilevati da `ConflictManager`.
2. **Refresh fisico** — `ScheduleRefresher.refreshMultiple` ricalcola tempi di transito e fermata.
3. **Allineamento Takt** — Se almeno un nodo ha `taktMinutes`, `TaktEngine.generaOrarioCadenzato` allinea arrivi/partenze all'hub; GA disabilitato in modalità Takt.
4. **AI cloud** — Opzionale (useAI); delega a `ScheduleAIResolver` e `RailwayAIService`.
5. **GA** — `GeneticOptimizer` con 250 iterazioni (solo se non Takt).
6. **Verifica** — Secondo refresh + report conflitti residui.

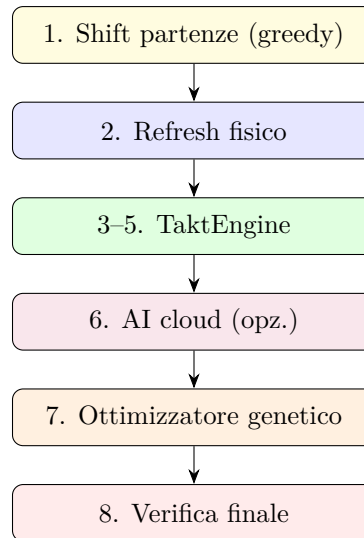


Figura 4.1: `ScheduleOptimizationPipeline.execute`.

4.4 Entry point

```

// ScheduleGenerationEngine (semplificato)
let topology = RailwayTopology(nodes: network.nodes, edges: network.edges)
let pipeline = ScheduleOptimizationPipeline(topology: topology)
return await pipeline.execute(
  newTrains: trains,
  existingTrains: existing,
  useAI: false,
  useGA: useGA,
  preferredTaktNodeId: taktNode
)

```

4.5 Dipendenze esterne (adapter app)

- `ConflictManager` — via `ScheduleConflictDetecting`
- `GeneticOptimizer` — via `ScheduleGeneticOptimizing`
- `RailwayAIService` — via `ScheduleAIOptimizing`
- `FDCSchedulerEngine` — via `TrainTravelTimeCalculating`

I protocolli sono definiti in `Domain/Services/` (FDCDomain SPM).

4.6 Test di regressione

4.7 Storico

Il capitolo sostituisce la documentazione del legacy `RailwayScheduleOptimizer` (rimosso in commit 00db4c2). Per confronto algoritmico vedere anche `FdC Railway Manager/RailwayAlgorithmDocs.md`.

Tabella 4.2: Suite test scheduling (FdC Railway ManagerTests).

File	Copertura
ServicesSchedulingTests	Pipeline, refresher, path resolver
Taktfahrplan120MinTests	Cicli Takt 120 min, hub timing
TrackAssignmentTests	Assegnazione binari
EdgeMigrationTests	Migrazione modello Edge

Capitolo 5

Modulo Pathfinding

5.1 Responsabilità

Il pathfinding calcola percorsi minimi sulla topologia ferroviaria: Dijkstra, distanza cumulata, percorsi alternativi, ordinamento nodi per linea fisica. Dal refactor Fase 5 la logica risiede in `Domain/Pathfinding/NetworkPathfinder.swift` — puro Foundation, senza `NetworkModel`.

5.2 API principale

Tabella 5.1: Funzioni `NetworkPathfinder`.

Funzione	Scopo
<code>findShortestPath</code>	Dijkstra: sequenza nodi + distanza
<code>findPathEdges</code>	Percorso come lista di <code>Edge</code>
<code>calculatePathDistance</code>	Somma <code>edge.distance</code> su un path
<code>findAlternativePaths</code>	Varianti Rapido / Alternativo / Panoramico
<code>findLineEndpoints</code>	Capolinea (grado 1 nel sottografo)
<code>buildOrderedPath</code>	Ordinamento topologico nodi linea
<code>resolveLinePath</code>	Validazione path linea fisica

5.3 Integrazione

- `RailwayTopology` delega `findPathEdges` e `calculatePathDistance` a `NetworkPathfinder`.
- `NetworkModel+Pathfinding` resta come *facciata* per il layer UI (Strangler): inoltra tutte le chiamate al dominio.
- Rimosso `RailwayTopology.init(network:)` — nessuna dipendenza `Domain` → `NetworkModel`.

5.4 Vincoli algoritmici

- Binario singolo non accoppiato: tratto **non direzionale**.
- Coppie direzionali (`pairedEdgeId`): rispetto del verso.
- `restrictIntermediateStations`: Dijkstra non espande *attraverso* stazioni intermedie (solo origine).

5.5 Test

`PathfindingTests` (target app): percorso diretto $A \rightarrow B$ e delega da `RailwayTopology`.

5.6 Evoluzione (Fase 6+)

Inclusione in `Packages/FDCDomain` quando `Node` sarà decoupled da `.localized`. Pathfinding è già Foundation-only.

Capitolo 6

Integrazione Swift Package

6.1 FDCDomain nel target Xcode

Dal refactor Fase 6 il package locale `Packages/FDCDomain` è collegato al target *FdC Railway Manager*:

- `XCLocalSwiftPackageReference` con `relativePath`
- `packageProductDependencies` + fase `Frameworks`
- `PBXFileSystemSynchronizedBuildFileExceptionSet`: i sorgenti in `Domain/Model/` e `Domain/Pathfinding/` non vengono ricompilati nell'app (evita simboli duplicati).
- `Models/DomainBridge.swift` con `@_exported import FDCDomain` e `typealias` per compatibilità con il codice esistente.

6.2 Decoupling UI (Fase 6)

Tabella 6.1: Spostamenti layer UI.

Prima	Dopo
<code>Node.localizedName</code>	<code>Models/Node+UI.swift</code>
<code>TrainDatabaseManager + View</code>	<code>Infrastructure/ + UI/Views/TrainDatabase/</code>
<code>TrainDatabaseModels (SwiftUI)</code>	Solo struct <code>Codable</code> in <code>Domain</code>

6.3 API pubbliche

I tipi esposti dal package richiedono `public` su proprietà e metodi usati dall'app, ad esempio:

- `RailwayTopology.nodes/edges`
- `Node.getTracksByProvenance, defaultVisualType`
- `Train.getPreferredTracks`
- `NetworkPathfinder` (tutti gli entry point statici)

6.4 Test

- **5 test** in `FDCDomainTests` (package SPM)
- **24 test** nel target app (inclusi `PathfindingTests`)

6.5 Prossimi passi (Fase 8)

- Estrarre `FDCScheduling` con protocolli in `Domain/Services/`
- Capitolo LaTeX `06-network-adapter.tex` completato in Fase 7

Capitolo 7

NetworkModel come adapter UI

7.1 Obiettivo (Fase 7)

Separare la **shell osservabile** usata dall'interfaccia dalla logica di query sulla topologia. `NetworkModel` resta nel target app; le operazioni pure vivono in **FDCDomain**.

7.2 NetworkTopologyService

`NetworkTopologyService` (`Domain/Services/`) è una struct `Sendable` che incapsula `RailwayTopology` e offre:

- lookup: `findNode`, `findEdge`
- filtri: `allStations`, `allJunctions`
- ordinamento: `sortedNodes`, `sortedEdges`
- connettività: `getConnectedNodeIds`, `getNeighborStations`
- geometria: `calculateDistance` (hub-aware, km)

Nessuna mutazione, nessuna dipendenza da SwiftUI o undo.

7.3 NetworkModel

`NetworkModel` delega le query a `NetworkTopologyService` tramite una computed property `topologyService`. Espone `topology: RailwayTopology` per i servizi di scheduling.

Le mutazioni (aggiunta nodi, binari, undo) restano in `Models/Network/NetworkModel+Updates.swift`.

7.4 DomainBridge

`DomainBridge.swift` espone il package e risolve l'ambiguità `Edge` (SwiftUI vs dominio):

```
@_exported import FDCDomain
public typealias Edge = FDCDomain.Edge
```

I typealias legacy per gli altri tipi sono stati rimossi.

7.5 TrainCategory e DTO

- `TrainCategory` — enum `Foundation in Domain/Model/`; estensione `SwiftUI` in `Models/TrainCategory+UI`
- `RailwayNetworkDTO` — struct `Codable` in `Infrastructure/` (I/O JSON, non dominio puro)

7.6 Test

- **2 test** aggiuntivi in `PathfindingTests` (app)
- **2 test** in `FDCDomainTests` per `NetworkTopologyService`

7.7 Prossimi passi (Fase 8)

- Estrarre `FDCScheduling` in package `SPM`
- Protocolli in `Domain/Services/` per `ConflictManager`, `GA`, `AI`

Capitolo 8

Package FDCScheduling

8.1 Obiettivo (Fase 8)

Estrarre la pipeline di scheduling in **FDCScheduling** SPM, collegata a **FDCDomain** tramite protocolli in `Domain/Services/`.

8.2 Architettura dipendenze

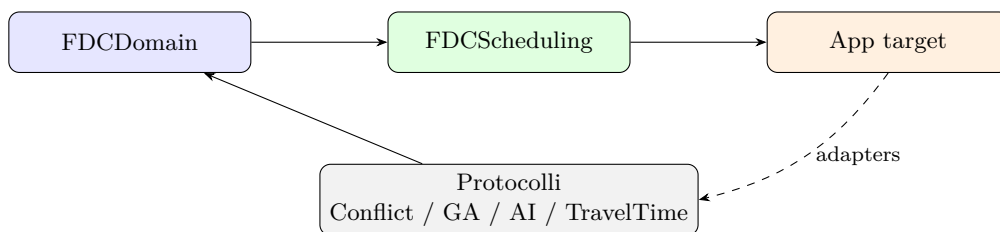


Figura 8.1: Dipendenze Fase 8: package puro + adapter nell'app.

8.3 Contenuto package

Symlink da `Services/Scheduling/` (7 file):

- `PathResolver`, `KinematicCalculator`
- `TaktEngine`, `TaktEngineScheduleGeneration`
- `ScheduleRefresher`, `VehicleSuitabilityEngine`
- `ScheduleOptimizationPipeline`

8.4 Protocolli (FDCDomain)

8.5 Shell app

Restano nel target app:

- `ScheduleGenerationEngine` — orchestratore UI
- `ScheduleAIResolver` — `RailwayAIService`

Tabella 8.1: Boundary tra package e app.

Protocollo	Implementazione app
ScheduleConflictDetecting	ConflictManagerAdapter
ScheduleGeneticOptimizing	GeneticOptimizerAdapter
ScheduleAIOptimizing	ScheduleAIResolverAdapter
TrainTravelTimeCalculating	FDCSchedulerTravelTimeAdapter

- AppSchedulingAdapters.swift — conformità protocolli
- SchedulingBridge.swift — @_exported import FDCScheduling

8.6 Test

- 6 test FDCDomainTests
- 4 test FDCSchedulingTests
- 24 test app (invariati, stub travel-time nei test)

8.7 Nota tecnica

I path con carattere + nelle membershipExceptions del pbxproj corrompono il parser Xcode; i file sono stati rinominati (es. DateScheduling.swift, TaktEngineScheduleGeneration.swift).

Appendice A

Indice dei sorgenti

Percorso	Modulo
Domain/Services/	NetworkTopologyService
Domain/Pathfinding/	NetworkPathfinder
Domain/Model/	FDCDomain
Services/Scheduling/	FDCScheduling
Editor/	Editor (<i>da documentare</i>)
Scheduler/	UI Scheduler
Simulator/	Simulator (<i>da documentare</i>)
RailwayAIService/	AI (<i>da documentare</i>)
Infrastructure/	I/O (<i>da documentare</i>)
Packages/FDCDomain/	SPM Domain
Packages/FDCScheduling/	SPM Scheduling